📄 Exploratory Data Analysis & Distributional Diagnostics

Module: Phase 1 | Week 6 (Days 1 & 2)
Subject: Univariate Analysis, Higher-Order Statistical Moments, Q-Q Plots, and Logarithmic Transformations

---

📑 Table of Contents

1. 🎯 Executive Summary & The Prerequisite of Distributional Parity
2. ⚙️ Univariate Distributional Diagnostics
   - 2.1 Higher-Order Statistical Moments: Skewness (γ_1)
   - 2.2 Tail-Heaviness & Outlier Propensity: Kurtosis (β_2)
3. 📉 Visual Diagnostics: Probability Density & Q-Q Plots
   - 3.1 Kernel Density Estimation (KDE) Mechanics
   - 3.2 Quantile-Quantile (Q-Q) Plot Verification
4. 🧱 Power Transformations: Normalizing Skewed Manifolds
   - 4.1 Logarithmic Scaling (`log1p`) vs. Box-Cox
   - 4.2 Restoring Homoscedasticity for Optimization Algorithmic Stability
5. 📋 Production Code Specification

🎯 1. Executive Summary & The Prerequisite of Distributional Parity

The primary objective of Week 6, Days 1 and 2, was to establish the diagnostic procedures required to audit raw continuous features before model training. Classical Machine Learning algorithms (such as Ordinary Least Squares Linear Regression, Logistic Regression, and Linear Discriminant Analysis) as well as gradient-optimized Neural Networks rely on a foundational mathematical assumption: **Normality (Gaussianity)**.

Real-world financial, operational, and demographic data sets are inherently asymmetric, frequently following power-law or lognormal distributions with long right-hand tails. Passing un-transformed, heavily skewed features into a parameterized model forces the loss function to be dominated by extreme values, introduces **heteroscedasticity** (non-constant error variance across predictions), and degrades model generalization. This phase focused on mathematically calculating the 3rd and 4th statistical moments (Skewness and Kurtosis), deploying visual Quantile-Quantile (Q-Q) validation, and executing non-linear power transformations to force skewed features into near-perfect Gaussian manifolds.

⚙️ 2. Univariate Distributional Diagnostics

Univariate analysis inspects each feature in isolation to determine its distributional parameters before evaluating feature-to-target interactions.

2.1 Higher-Order Statistical Moments: Skewness (γ_1)

While the 1st and 2nd statistical moments represent the Mean (μ) and Variance (σ^2), the 3rd standardized moment is **Skewness (γ_1)**, quantifying the asymmetry of the probability distribution about its mean:

$$\gamma_1 = E\left[\left(\frac{X-\mu}{\sigma}\right)^3\right] = \frac{\frac{1}{N}\sum_{i=1}^{N}(x_i-\bar{x})^3}{\left(\frac{1}{N}\sum_{i=1}^{N}(x_i-\bar{x})^2\right)^{3/2}}$$

- **Symmetric ($\gamma_1 \approx 0$):** Mean, Median, and Mode align. Optimal for linear and distance-based estimators.
- **Positively Skewed ($\gamma_1 > 1.0$):** The right tail is elongated. The Mean is dragged significantly higher than the Median (standard in income, transaction amounts, and real estate data).
- **Negatively Skewed ($\gamma_1 < -1.0$):** The left tail is elongated. The Mean is dragged lower than the Median (standard in asset depreciation and lifespan metrics).

2.2 Tail-Heaviness & Outlier Propensity: Kurtosis (β_2)

The 4th standardized moment is **Kurtosis**, quantifying the thickness of the distribution tails relative to a normal distribution:

$$\beta_2 = E\left[\left(\frac{X-\mu}{\sigma}\right)^4\right] = \frac{\frac{1}{N}\sum_{i=1}^{N}(x_i-\bar{x})^4}{\sigma^4}$$

- **Mesokurtic ($\beta_2 = 3.0$, Fisher's Excess Kurtosis $= 0$):** Standard normal distribution tail weight.

- **Leptokurtic (Excess Kurtosis > 0):** Sharp central peak with heavy, fat tails. Signals high outlier frequency that will destabilize standard L_2 norm loss functions (Mean Squared Error).
- **Platykurtic (Excess Kurtosis < 0):** Flat peak with thin tails, indicating light outlier presence.

📉 3. Visual Diagnostics: Probability Density & Q-Q Plots

Relying strictly on summary scalars can mask multimodal distributions (distributions with multiple peaks). Visual diagnostics are mandatory.

3.1 Kernel Density Estimation (KDE) Mechanics

A standard histogram bins continuous data arbitrarily, making the visual representation sensitive to the chosen bin width.
Kernel Density Estimation (KDE) solves this by placing a continuous Gaussian curve (a Kernel) over every individual data point in the array. The algorithm sums these micro-kernels together, generating a smooth, non-parametric Probability Density Function (PDF) that exposes true data concentration independent of binning artifacts.

3.2 Quantile-Quantile (Q-Q) Plot Verification

A Q-Q plot compares the quantiles of the experimental data against the theoretical quantiles of a mathematically perfect standard normal distribution ($\mu = 0, \sigma = 1$).

- **Interpretation:** The algorithm sorts the experimental data and plots it against the standard normal line. If the data is Gaussian, the plotted points fall along a straight 45-degree reference line.
- **Diagnosis:** If the points curve upward away from the line at the upper boundary, it proves severe positive skewness and fat-tailed kurtosis, providing empirical proof that a transformation is non-negotiable.

🧱 4. Power Transformations: Normalizing Skewed Manifolds

When extreme skewness ($\gamma_1 > 1.0$) is diagnosed, the pipeline executes a mathematical transformation to stabilize variance and restore symmetry.

4.1 Logarithmic Scaling (`log1p`) vs. Box-Cox

The most reliable transformation for positively skewed economic data is the natural logarithm:

$y = \ln(1 + x)$

- **The Zero-Value Protection (`log1p`):** Standard $\ln(x)$ is undefined for $x = 0$ ($\lim_{x \to 0^+} \ln(x) = -\infty$). Utilizing `np.log1p(x)` computes $\ln(1 + x)$, ensuring zero-value transactions map to $\ln(1) = 0$, completely preventing runtime mathematical exceptions.
- **Box-Cox Transformation:** A generalized power transform with parameter λ:
  $y^{(\lambda)} = \begin{cases} \frac{x^\lambda - 1}{\lambda} & \text{if } \lambda \neq 0 \\ \ln(x) & \text{if } \lambda = 0 \end{cases}$
  Box-Cox uses maximum likelihood estimation to determine the optimal λ automatically, but strictly requires $x > 0$. For zero-inclusive data, `np.log1p` or the **Yeo-Johnson** transformation is required.

4.2 Restoring Homoscedasticity for Optimization Algorithmic Stability

Why does normalization matter to Machine Learning?

1. **Gradient Descent Acceleration:** In skewed feature spaces, the gradients computed during backpropagation are erratic—massive for tail values and tiny for central values. Normalizing the feature contracts the dynamic range, creating a spherical loss surface that allows Gradient Descent to step directly toward the global minimum.
2. **Error Homoscedasticity:** Linear models assume errors ($\epsilon = y - \hat{y}$) have constant variance across all levels of the target. Un-transformed skewed targets create funnel-shaped residuals (heteroscedasticity), invalidating statistical confidence intervals and p-values. Log transformations stabilize the variance across the entire target domain.

📋 5. Production Code Specification

Below is the verified diagnostic script executing univariate analysis, higher-order statistical moment calculation, and dual Q-Q plot validation.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from scipy import stats
from pathlib import Path

def run_univariate_diagnostics():
    root = Path(__file__).resolve().parent
    data_dir = root / "data"
    output_plot = data_dir / "univariate_transformation.png"

    # 1. Synthesize Lognormal Right-Skewed Data
    np.random.seed(42)
    raw_incomes = np.random.lognormal(mean=2.5, sigma=1.0, size=5000)
    df = pd.DataFrame({"income_lakhs": raw_incomes})

    # 2. Moment Computation (Pre-Transformation)
    raw_skew = stats.skew(df["income_lakhs"])
    raw_kurt = stats.kurtosis(df["income_lakhs"]) # Fisher definition
(normal = 0)

    # 3. Log1p Transformation Execution
    df["income_log"] = np.log1p(df["income_lakhs"])
    trans_skew = stats.skew(df["income_log"])
    trans_kurt = stats.kurtosis(df["income_log"])

    # 4. Multi-Panel Diagnostic Dashboard
    sns.set_theme(style="whitegrid")
    fig, axes = plt.subplots(2, 2, figsize=(14, 10))

    # Raw Distribution
    sns.histplot(df["income_lakhs"], kde=True, color="#e74c3c",
ax=axes[0, 0], bins=50)
    axes[0, 0].set_title(f"Raw Income (Skew: {raw_skew:.2f})",
fontweight="bold")
```

```
    # Raw Q-Q Plot
    stats.probplot(df["income_lakhs"], dist="norm", plot=axes[0, 1])
    axes[0, 1].set_title("Raw Q-Q Plot (Non-Normal Deviation)",
fontweight="bold")

    # Transformed Distribution
    sns.histplot(df["income_log"], kde=True, color="#2ecc71", ax=axes[1,
0], bins=50)
    axes[1, 0].set_title(f"Transformed log1p (Skew: {trans_skew:.2f})",
fontweight="bold")

    # Transformed Q-Q Plot
    stats.probplot(df["income_log"], dist="norm", plot=axes[1, 1])
    axes[1, 1].set_title("Transformed Q-Q Plot (Normal Alignment)",
fontweight="bold")

    plt.tight_layout()
    plt.savefig(output_plot, dpi=300)
    plt.close()

if __name__ == "__main__":
    run_univariate_diagnostics()
```